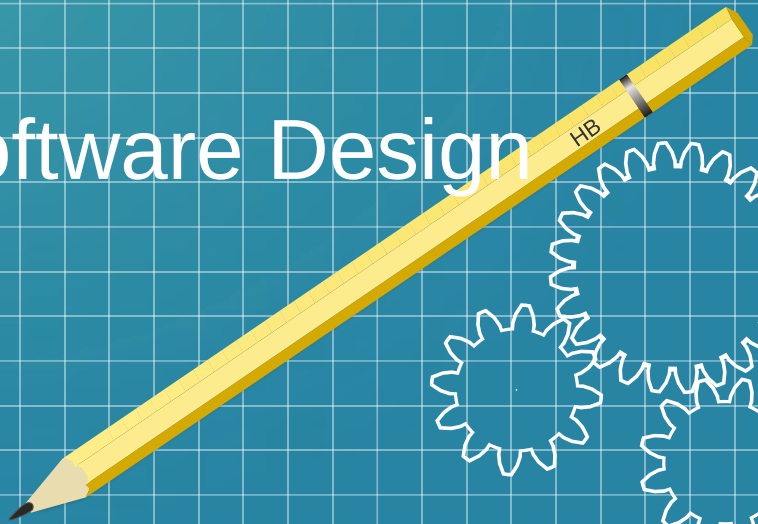
CSC-300: Software Engineering

UNIT-III: Function Oriented Software Design



Function-Oriented Software Design

- In this design technique, **we view a system as a black-box that provides a set of *services* to the users of the software.**
- These services are also known as the ***high-level functions*** supported by the software.
- **During the design process, these high-level functions are successively decomposed into more detailed functions.**

Function-Oriented Software Design

- ***Top-down decomposition*** **denotes the successive decomposition of a set of high-level functions into more detailed functions.**
- **Then the different identified functions are mapped to modules and a module structure is created.**

Overview of SA/SD Methodology

- SA/SD methodology involves carrying out two distinct activities:
  - **Structured Analysis (SA)**
  - **Structured Design (SD)**

Structured Analysis - *Introduction*

- **During structured analysis, the SRS document is transformed into a data flow diagram (DFD) model.**
- **During structured analysis, functional decomposition of the system is achieved. Each function that the system needs to perform is analysed and hierarchically decomposed into more detailed functions.**

Structured Analysis – *Data Flow Diagrams (DFDs)*

- **A DFD is a hierarchical graphical model of a system that shows the different processing activities or functions that the system performs and the data interchange among those functions.**
- DFD model only represents the data flow aspects **it completely ignores aspects such as control flow, the specific algorithms used by the functions, etc.**
- In the DFD terminology, each function is called a ***process*** or a ***bubble***.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Symbols*

- There are essentially five different types of symbols used for constructing DFDs.

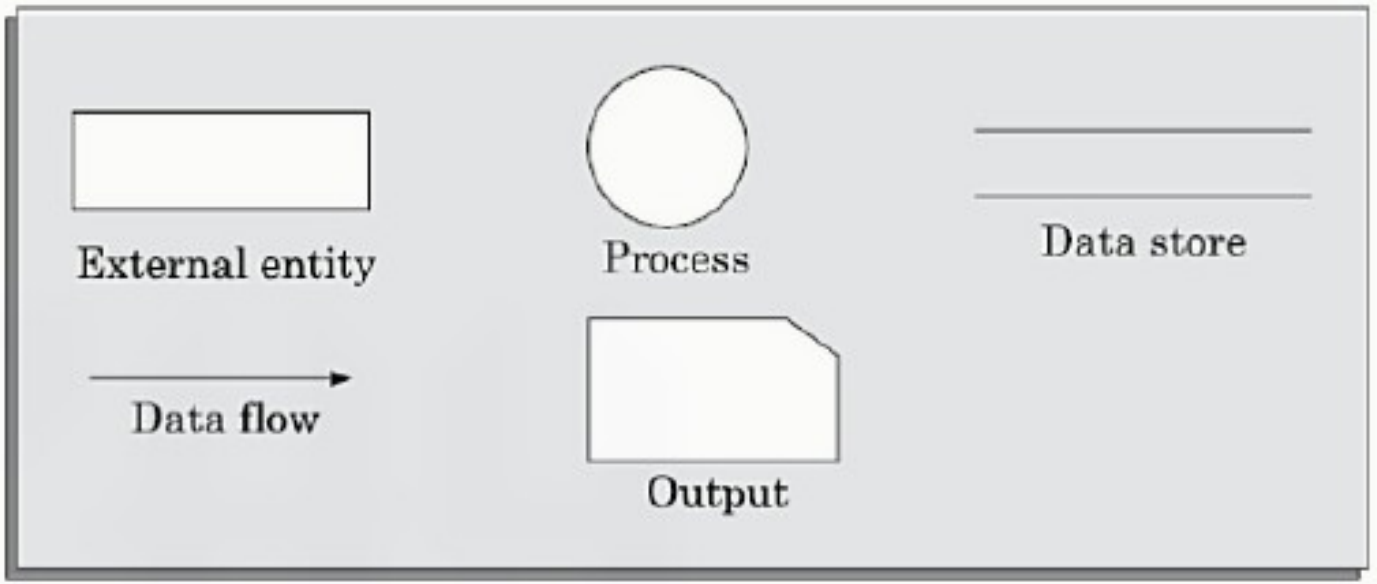


Figure 6.2: Symbols used for designing DFDs.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Symbols (contd.)*

- **Function symbol**: **A function is represented using a circle**. This symbol is called a process or a bubble. **Bubbles are annotated with the names of the corresponding functions.**
- **External entity symbol**: The external entities are **physical entities external to the software system which interact by inputting data to the system or by consuming the data produced by the system.** The external entity symbols can be used to represent **external hardware and software**. **It is represented by a rectangle**.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Symbols (contd.)*

- **Data flow symbol**: **A directed arc (or an arrow) is used as a data flow symbol.** It **represents the data flow occurring between two processes or between an external entity and a process in the direction of the data flow arrow.** Data flow symbols are **usually annotated with the corresponding data names.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Symbols (contd.)*

- **Data store symbol: A data store is represented using two parallel lines. A data store symbol can represent either a data structure or a physical file on disk. The direction of the data flow arrow shows whether data is being read from or written into a data store.**
- **Output symbol: The output symbol is used when a hard copy is produced.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Synchronous operations*

- **Synchronous operations**
  - If two bubbles are directly connected by a data flow arrow, then they are ***synchronous***.
  - It means the two processes operate at the same speed.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Asynchronous operations*

- **Asynchronous operations**
  - If two bubbles are connected through a data store, then they are ***asynchronous***.
  - The speed of the operations of the bubble are independent.

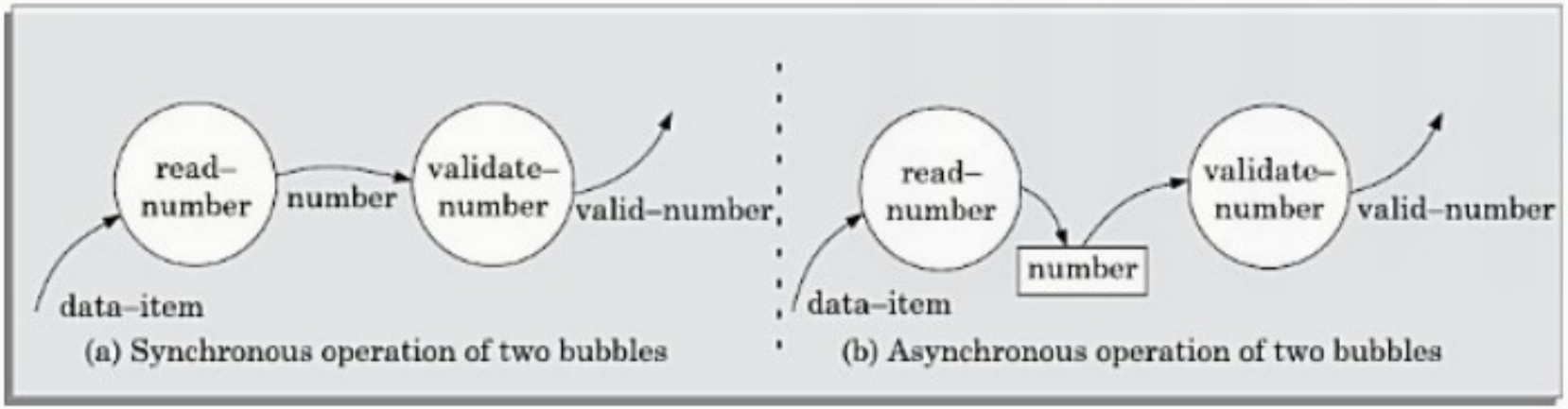


Figure 6.3: Synchronous and asynchronous data flow.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Data Dictionary*

- **Data Dictionary**
  - **A data dictionary lists all data items that appear in a DFD model**.
  - The data items listed include **all data flows and the contents of all data stores appearing on all the DFDs**.
  - It **lists the purpose of all data items and the definition of all composite data items** in terms of their component data items.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Data Dictionary (contd.)*

- **Data Dictionary**
  - A data dictionary **provides a standard terminology** for all relevant data for use by the developers working in a project.
  - Helps the developers **to determine the definition of different data structures** in terms of their component elements
  - The data dictionary helps **to perform impact analysis**.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Data definition*

- **Data definition**

  Composite data items can be defined in terms of primitive data items

  - **+** denotes composition of two data items, e.g. a+b represents data a and b.
  - **[,,]** represents selection, i.e. any one of the data items listed inside the square bracket can occur
  - **()** the contents inside the bracket represent optional data which may or may not appear.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Data definition (contd.)*

- **Data definition**
  - **{}** represents iterative data definition, e.g. {name}5 represents five name data. {name}* represents zero or more instances of name data.
  - **=:** represents equivalence, e.g. a=b+c means that a is a composite data item comprising of both b and c.
  - ***/* */*** Anything appearing within is considered as comment.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing*

- The DFD model of a system is constructed by using a hierarchy of DFDs. **The top level DFD is called the level 0 DFD or the *context diagram***. This is the most abstract (simplest) representation of the system (highest level).
- **To develop a higher-level DFD model, processes are decomposed into their subprocesses and the data flow among these subprocesses are identified.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Context Diagram*

- Context diagram **represents the entire system as a single bubble**.
- The **bubble is annotated with the name of the software system being developed (usually a noun)**.
- **It establishes the context in which the system operates**; that is, **who are the users**, **what data do they input to the system**, and **what data they received by the system**.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Context Diagram*

- The various external entities with which the system interacts and the **data flow occurring between the system and the external entities are represented**.
- These **data flow arrows should be annotated with the corresponding data names**.

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Level 1 DFD*

- The **level 1 DFD usually contains three to seven bubbles**.
- To develop the level 1 DFD, **examine the high-level functional requirements in the SRS document**.
- Next, **examine the input data to these functions and the data output by these functions** as documented in the SRS document

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Level 1 DFD*

- **If a system has more than seven high-level requirements some of the related requirements have to be combined and represented as a single bubble**
- **If a system has less than three high-level functional requirements, some of the high-level requirements need to be split into their subfunctions.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Decomposition*

- Decomposition of a bubble is also known as ***factoring*** or ***exploding*** a bubble.
- **Each bubble at any level of DFD is usually decomposed to anything three to seven bubbles**.
- A bubble **should not be decomposed any further once a bubble is found to represent a simple set of instructions**.
- For **simple problems, decomposition up to level 1 should suffice**.
- However, **large industry standard problems may need decomposition up to level 3 or level 4.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Numbering*

- Numbering **help in uniquely identifying any bubble in the DFD**.
- **Bubble at the context level is usually assigned the number 0**
- **Bubbles at level 1 are numbered, 0.1, 0.2, 0.3, etc.**
- **When a bubble numbered x is decomposed, its children bubble are numbered x.1, x.2, x.3, etc.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Balancing*

- **The data that flow into or out of a bubble must match the data flow at the next level of DFD.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Things to note*

- **All external entities interacting with the system should be represented only in the context diagram. The external entities should not appear in the DFDs at any other level.**
- **Only three to seven bubbles per diagram should be allowed.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Shortcomings*

- **Imprecise DFDs leave ample scope to be imprecise.** For example, the find-book-position bubble may not convey anything regarding what happens when the required book is missing.
- **The order in which inputs are consumed and outputs are produced by a bubble is not specified. A DFD model does not specify the order in which the different bubbles are executed. Representation of such aspects is very important for modelling real-time systems.**

Structured Analysis – *Data Flow Diagrams (DFDs) -- Developing --- Shortcomings*

- **The method of carrying out decomposition to arrive at the successive levels and the ultimate level to which decomposition is carried out are highly subjective** and depend on the choice and judgment of the analyst.

Structured Design

- During structured design, **the DFD model is transformed into a structure chart**.
- **All functions identified during structured analysis are mapped to a module structure. This module structure is also called the *high-level design* or the *software architecture* for the given problem, represented using *structure chart***.
- It is normally **followed by a detailed design stage**.

Structured Design – *Making of a structure chart*
*-- **Symbols***

- **Rectangular boxes**: **A rectangular box represents a module.** Usually, every rectangular box is **annotated with the name of the module**.
- **Module invocation arrow**s: **An arrow connecting two modules implies that during program execution control is passed from one module to the other in the direction of the connecting arrow.** The frequency and the order in which the modules are invoked is not relevant to the structure chart.

Structured Design – *Making of a structure chart*
-- Symbols (contd.)

- **Data flow arrows**: Data flow arrows **represent the named data passes from one module to the other in the direction of the arrow.**
- **Library modules**: **A library module is usually represented by a rectangle with double edges.** Libraries **comprise the frequently called modules. When a module is invoked by many other modules, it is made into a library module.**

Structured Design – *Making of a structure chart*
-- Symbols (contd.)

- **Selection**: The **diamond symbol represents the fact that one module of several modules connected with the diamond symbol is invoked depending on the outcome of the condition attached with the diamond symbol**.
- **Repitition**: **A loop around the control flow arrows denotes that the respective modules are invoked repeatedly.**

Structured Design – *Making of a structure chart*
-- Rules

- There **should be one and only one module at the top, called the *root***.
- **There should be at most one module invocation arrow between any two modules**, i.e., if module A invokes module B, module B cannot invoke module A.

Structured Design – ***Flow chart vs Structure chart***

- It is usually **difficult to identify the different modules of a program from its flow chart representation**.
- **Data interchange among different modules is not represented in a flow chart**.
- **Sequential ordering of tasks that is inherent to a flow chart is suppressed in a structure chart**.

Structured Design – ***Detailed Design***

- During detailed design **the pseudo code description of the processing and the different data structures are designed for the different modules of the structure chart**.
- They are described in the form of **module specification (MSPEC)**.

Structured Design – *Detailed Design -- **MSPEC***

- MSPEC is **usually written using structured English**.
- The MSPEC for the **non-leaf modules describe the different conditions under which the responsibilities are delegated to the lower-level modules**.
- The **MSPEC for the leaf-level modules should describe in algorithmic form how the primitive processing steps are carried out**.

Structured Design – ***Design Review***

- After a design is complete, the design is required to be reviewed.
- **The review team usually consists of members with design, implementation, testing, and maintenance perspectives**, who may or may not be the members of the development team.

Structured Design – *Design Review (contd.)*

- The review team checks the design documents especially for the following aspects:
  - **Traceability**: **Whether each bubble of the DFD can be traced to some module in the structure chart and vice versa. Also, whether each functional requirement in the SRS document can be traced to some bubble in the DFD model and vice versa**.

Structured Design – *Design Review (contd.)*

- **Correctness: Whether all the algorithms and data structures of the detailed design are correct.**
- **Maintainability: Whether the design can be easily maintained in future.**
- **Implementation: Whether the design can be easily and efficiently be implemented.**